FYP REPORT WRITING GUIDE

What Every Chapter Must Contain

A structured reference for students preparing their Final Year Project report

Lee Kong Chian Faculty of Engineering and Science | Software Engineering

Report Structure Overview

Your FYP report consists of 8 chapters — each with a distinct purpose and required content.

This structure follows the LKCFES FYP Report Template for Development-Based projects. Front-matter, formatting, and referencing rules below apply to ALL FYP types.

1. **Introduction**
   Context, problem, goals, scope
2. **Literature Review**
   Existing work, methodology, technologies
3. **Methodology & Work Plan**
   SDLC approach, WBS, Gantt chart, tools
4. **Project Specification**
   Requirements, fact-finding, UML diagrams
5. **System Design**
   UI flow, architecture, DB schema
6. **System Implementation**
   Frontend, backend, API integration
7. **System Testing**
   UI, unit, API, integration, UAT
8. **Conclusion**
   Achievement, limitations, future work

Front Matter & Declaration Requirements

Declaration Statement — Declare the report is original work, fully acknowledged. Tick the relevant boxes: (1) if GenAI tools (ChatGPT, Gemini, Copilot, etc.) were used for language refinement, and/or (2) if vibe-coding tools (Replit Agent, Cursor Composer, Bolt, etc.) were used in development. Sign with Name, ID No., and Date.

Abstract — A single paragraph of no more than 300 words, following the IMRaD structure (Introduction, Method, Results, Discussion). Must summarise the problem, methodology, key outcomes, and conclusions.

Keywords — Provide at least 5 relevant keywords describing your work (e.g., mobile application development; API integration; recommendation system).

Subject Area — Identify the Library of Congress subject classification for your project via the UTAR Library catalogue (e.g., QA76.75–76.765 Computer Software).

Length of Project Report — FYP1: < 10,000 words.

FYP2: < 30,000 words.

Examiner Tip

Students remain fully responsible for the accuracy and integrity of the report — declaring GenAI/vibe-coding tool usage does NOT shift responsibility for falsified or misrepresented results. Starting June 2026, the AI writing report requirement is <20%.

FYP1 vs FYP2 — What Changes

Acknowledgements — NOT required in the FYP1 report. Optional in the FYP2 report.

Abstract — NOT required in FYP1. Compulsory in FYP2.

1.6 Contribution of the Study — NOT required in FYP1. Compulsory in FYP2 — explain the value your completed project adds to users, industry, or knowledge.

1.7 Outline of the Report — NOT required in FYP1. Compulsory in FYP2 — a short paragraph summarising what each subsequent chapter covers.

Examiner Tip

Always confirm with your supervisor which version (FYP1 / FYP2) of the template applies to your current submission — section requirements differ.

CHAPTER 1

Introduction

Setting the stage — why your project matters

What Must Be in Chapter 1

1.1 General Introduction — Provide background on the problem domain. Explain the context and relevance of your project with supporting citations.

1.2 Importance of the Study — Explain why this project matters — who benefits (users, industry, organisation) and how, if the problem is successfully addressed.

1.3 Problem Statement — Identify and describe specific, real problems. Break it into sub-problems (e.g., time-consuming research, limited personalisation, budget tracking). End with a summary.

1.4 Aim and Objectives — State the overall aim in 1–2 sentences, then list 3–5 SMART objectives that are measurable outcomes with target dates — NOT activities or deliverables.

Examiner Tip

Objectives must be outcome-based. ✗ 'Develop a system' is a deliverable. ✓ 'Identify and analyse existing applications by [date]' is an objective.

Chapter 1 — Continued

1.5 Scope and Limitation of the Study — Define what is included and excluded — target users, platform (mobile/web), features covered, and known constraints. Dev-based projects commonly extend this with a Proposed Solution and Proposed Approach (high-level system description, methodology, and technology stack).

1.6 Contribution of the Study (FYP2 only) — Explain the value the completed project brings — to users, the organisation, or the body of knowledge.

1.7 Outline of the Report (FYP2 only) — A short paragraph summarising what each subsequent chapter covers.

Front Matter Check — Ensure Declaration, Abstract, Table of Contents, List of Tables/Figures, List of Symbols/Abbreviations, and List of Appendices are complete and correctly paginated.

Examiner Tip

The Abstract is often the first thing an examiner reads. It must summarise the problem, approach, technologies, and outcome in 500 words or fewer (check the exact number with your supervisor), following IMRaD structure.

CHAPTER 2

Literature Review

Establishing what is already known — and where the gap is

What Makes a Literature Review “Critical”

Not a Compilation — The review must NOT simply compile or reproduce other people's work. You are expected to examine and comment critically on literature relevant to your research area.

Synthesis Is the Real Skill — Finding the right sources is only a small part of the process. Success depends on integrating ideas, information, data, and arguments into a coherent, well-reasoned discussion.

Depth Over Breadth — A review of around half a dozen of the most important works — discussed in depth — plus brief comments on a few minor ones is usually sufficient. Quality of analysis matters more than quantity of sources.

Show the Diversity of Views — Clearly bring out where authors agree, disagree, or take different approaches.

Position Your Own Research — End by showing how and where your project fits into this landscape — what gap it addresses or how it builds on existing work.

Examiner Tip

Before diving into application/technology comparisons, ask of every source: What does this argue? How does it relate to other sources? Where does my project sit in relation to it?

What Must Be in Chapter 2

2.1 Introduction — Briefly describe what this chapter covers — existing applications, methodology review, and technology review.

2.2 Existing / Similar Application Research — Study and compare at least 6–9 existing systems. Use a feature comparison table. Identify strengths and weaknesses of each. Conclude with a summary of gaps your project will fill.

2.3 Software Development Methodology — Review at least 3 methodologies (e.g., Waterfall, RAD, Agile, Spiral). Include a comparison table and justify your selection based on project characteristics.

Examiner Tip

Apply the critical mindset here too: don't just list features side-by-side — discuss what the differences reveal, and end each section by stating how it shapes your design decisions.

Chapter 2 — Technology Reviews

2.4 Frontend Framework Review — Compare at least 2–3 frameworks (e.g., React Native, Flutter, Ionic). Discuss similarities and differences. Conclude with a justified selection.

2.5 Backend Framework Review — Review backend options (e.g., Node.js, Django, Spring Boot), database systems (MySQL, MongoDB, Firebase), and any AI/ML integration libraries. Justify each selection.

Citation Requirement — Every claim must be supported by a citation. Use CTR Harvard Referencing Style. Avoid relying on a single source; aim for diversity of references.

Comparison Tables — Use tables to compare frameworks or applications systematically. Rows = criteria (e.g., performance, scalability, cost). Columns = options.

Examiner Tip

Aim for at least 20 references, mostly from journals and conference papers (within the past 5 years). Wikipedia is NOT an acceptable source.

REFERENCING — APPLIES TO ALL CHAPTERS

Referencing: CTR Harvard Style

Faculty Standard — LKCFES adopts the Cite Them Right (CTR) Harvard Referencing Style for all citations and references throughout the report.

Two Required Components — Every source needs an in-text citation at the point of use, and a full reference entry in the Reference List at the end of the report.

Journal Article, 4+ Authors — Reference: Author, Initial. et al. (Year) 'Title of article', Title of Journal, volume(issue), pp. xx-xx. In-text: Norrie et al. (2012) hold the view that...

Electronic Journal Article, 2 Authors — Include the DOI or URL and an access date for online sources. In-text: Barke and Mowl's (2016) study found that...

Examiner Tip

Consistency matters more than quantity. Every in-text citation must have a matching Reference List entry, and every entry must follow CTR Harvard formatting exactly — check the UTAR Library's CTR Harvard guide.

CHAPTER 3

Methodology & Work Plan

How you will build the project — and your timeline

What Must Be in Chapter 3

3.1 Introduction — Summarise what this chapter covers: methodology selection, work breakdown, project schedule, and development tools.

3.2 Methodology — State selected methodology. Describe your modified/adapted version of it. Explain each phase (e.g., Planning, Prototyping, Testing, Deployment) with what happens in each phase for your specific project.

3.3 Work Plan — Include a Work Breakdown Structure (WBS) that decomposes all major tasks. Include a project schedule table with start/end dates per task. Include a Gantt Chart showing the full timeline.

3.4 Development Tools — Describe every tool used (IDE, project management, design, VCS, testing). Include screenshots, version numbers, and justify each selection.

Examiner Tip

If you claim to use Agile, you must show sprint evidence — sprint backlog, sprint reviews, or iteration plans. Declaring Agile without operationalising it is a critical examiner concern.

CHAPTER 4

Project Specification

What the system must do — from user needs to UML models

What Must Be in Chapter 4

4.1 Introduction — State that this chapter covers fact-finding results, software requirements, and UML models.

4.2 Fact Finding — Document your primary research — surveys, interviews, or observation. Include questionnaire screenshots, a summary of findings (charts/tables), and explicitly link the findings to system requirements.

4.3 Software Requirement Specification (SRS) — 4.3.1 Functional Requirements — list specific features the system must perform (tabulated, numbered). 4.3.2 Non-Functional Requirements — performance, security, usability, scalability constraints.

4.4 Unified Modelling Language (UML) — 4.4.1 Use Case Diagram — shows all actors and use cases. 4.4.2 Use Case Description — detailed description for every use case (actor, precondition, main flow, alternate flow, postcondition).

Examiner Tip

Fact-finding must come before requirements. Requirements must trace back to findings — show that your requirements are derived from user data, not invented.

The Prototype: A Required Part of Chapter 4

What Is a Prototype? — An early, preliminary version of the proposed system built to demonstrate how the final product is intended to look and/or work. It is NOT the finished system — it exists to test ideas, validate requirements, and surface design issues early.

Non-Functional Prototype — A clickable mock-up (e.g., built in Figma) that simulates the user interface and navigation flow WITHOUT real underlying logic or data processing. Buttons may link between screens, but no actual computation, storage, or backend processing occurs.

Functional Prototype — A partially working build with real code — at least some features actually operate (e.g., a working login, one CRUD module connected to a database). Demonstrates technical feasibility of the chosen tech stack, not just the look and feel.

Why FYP1 Requires It — The prototype provides early proof that the proposed solution is feasible and aligned with the SRS (Chapter 4.3) before full-scale development in FYP2. It also gives supervisors/examiners something concrete to assess.

Examiner Tip

Either type is acceptable — but it must be presented as evidence, not just described. Include annotated screenshots of every key screen/flow, and explicitly map each prototype screen back to a requirement or use case.

What Must Be in Chapter 4

The FYP 1 report should be completed up to Chapter 4 and is named as Project Initial Specification.

The purpose of the prototype is to visualize the proposed solution and provide evidence of the system design before full implementation in FYP 2.

In FYP 2, this chapter will be updated and renamed as Project Specification, reflecting the finalized system requirements and design based on the completed development and implementation of the project. You may also remove the contents on prototype.

Examiner Tip

Do not forget to update the chapter title and revise the contents accordingly in the final report submission.

CHAPTER 5

System Design

Translating requirements into a blueprint

What Can Be in Chapter 5

5.1 Introduction — Describe what this chapter covers: UI flow, UI design mock-ups, system architecture, database schema, and data dictionary.

5.2 User Interface Flow Diagram — Show how users navigate between screens. Every screen in the app should be represented. Use arrows to show transitions and conditional flows (e.g., login success vs. failure).

5.3 User Interface Design — Present Figma/prototype mock-ups for all major screens. Annotate design choices where relevant (colour, layout rationale).

5.4 System Architecture Diagram — Show how all components (frontend, backend, external APIs, database) interact. Clearly label technologies used at each layer.

Examiner Tip

Every design element should map directly to a requirement from Chapter 4. If a screen exists in your design, there must be a use case for it.

What Can Be in Chapter 5

5.5 Database Schema — Show an Entity-Relationship Diagram (ERD) or collection/table schema diagram. Identify all entities, attributes, primary keys, foreign keys, and relationships.

5.6 Data Dictionary — Provide a tabulated definition of every field in every table/collection — field name, data type, constraints, description. This must be complete and match your implementation.

5.7 Data Flow Diagram — Shows how data moves through your system.

5.8 Activity Diagram — Shows the workflow or sequence of activities performed to complete a task.

5.9 Summary — Briefly recap what was designed and how the design decisions support the project objectives.

Examiner Tip

The contents and subsections of Chapter 5 may differ from one project to another. Please check with your supervisor to ensure that all required sections are included in your report.

CHAPTER 6

System Implementation

Showing your system was actually built

What Must Be in Chapter 6

The purpose of this chapter is to describe how the design was translated into a working system.

6.1 Introduction — State what this chapter covers: project setup, frontend and backend implementation, and API integration.

6.2 Project Setup — Describe the initialisation steps — environment setup, project scaffolding commands, directory structure, dependency installation.

6.3 Frontend Implementation — For each module (e.g., for travel app planner - Auth, Itinerary, Budget, Hotel, Flight), include: a screenshot of the implemented screen and how the module fulfils its requirement.

6.4 Backend Implementation — Show server setup, data models (Mongoose/ORM schemas), and key API endpoints with request/response structure and code. Cover at least CRUD for your main entities.

Examiner Tip

Screenshots must show the actual working system — not the design mock-up. It is not necessary to include code snippets.

Chapter 6 — API Integration

6.5 API Integration — 6.5.1 Integration with Backend Server — show how frontend calls backend REST APIs (Axios/Fetch calls, error handling). 6.5.2 Integration with External APIs — document each third-party API including endpoint, data format, and sample response.

6.6 Summary — Summarise what was implemented and confirm alignment with design and requirements from Chapters 4 and 5.

Code Quality — Ensure code follows consistent naming conventions, has comments, and is modular. Avoid dumping hundreds of lines of uncommented code.

Traceability — Each module or feature described must correspond to a functional requirement listed in Chapter 4.3.

Examiner Tip

Examiners will compare your implemented screens against your design mock-ups. Unexplained differences between design and implementation must be addressed.

The contents and subsections of Chapter 5 may differ from one project to another. Please check with your supervisor to ensure that all required sections are included in your report.

CHAPTER 7

System Testing

Proving your system works correctly

What Must Be in Chapter 7

7.1 Introduction — State the types of testing conducted and the overall purpose: validating the system against user expectations and performance standards.

7.2 User Interface Testing — Early UI-only testing before backend integration. Distribute a questionnaire to testers (5+ participants). Present results using charts. Document feedback received and changes made.

7.3 Unit Testing — Test individual functions or components in isolation. Provide a test case table: Test ID, Description, Input, Expected Output, Actual Output, Pass/Fail.

7.4 API Testing — Use tools like Postman to test all backend endpoints. Document: HTTP method, endpoint URL, request body, expected response, actual response, status code.

Examiner Tip

Every test must show actual results — not just 'Pass'. Include screenshots of Postman results, test runners, or app screens for key test cases.

Chapter 7 — Integration & Acceptance Testing

7.5 Integration Testing — Test interactions between modules (frontend ↔ backend, system ↔ external APIs). Document end-to-end scenarios with expected and actual outcomes.

7.6 System Usability Testing (SUS) — Distribute a standardised usability questionnaire (e.g., System Usability Scale) to participants. Compute the SUS score and interpret the result. Show demographic data of participants.

7.7 User Acceptance Testing (UAT) — Final validation with real or representative users. Testers execute predefined scenarios. Record pass/fail per scenario. Present final acceptance decision.

Test Coverage — Ensure all functional requirements from Chapter 4.3 have at least one corresponding test case. Use a traceability matrix if needed.

Examiner Tip

For projects involving LLM-generated responses, additional testing might include response quality evaluation (e.g., relevance, accuracy, completeness, and clarity of responses), performance testing (e.g., response time), hallucination testing, and response consistency testing. The specific tests included should be selected based on the project's objectives and scope.

CHAPTER 8

Conclusion

Reflecting on what was achieved — and what comes next

What Must Be in Chapter 8

8.1 Introduction / Achievements — Restate the problem and explain how the project solved it. Map each project objective to its corresponding achievement. Explicitly confirm whether each objective was met (with evidence).

8.2 Limitations — Identify genuine constraints encountered — technical, time, resource, or design limitations. Do not omit this section. Examiners expect honest, reflective analysis. Each limitation should be specific and explained.

8.3 Future Improvements — Propose concrete, achievable enhancements based on the limitations identified. Each improvement should explain why it was not done now and how it would benefit the system.

Examiner Tip

Limitations are NOT a sign of a weak project — they demonstrate critical thinking. Vague statements like 'improve performance' are insufficient; be specific.

FORMATTING — APPLIES TO ALL CHAPTERS

Table & Figure Formatting Rules

Tables — Numbered sequentially per chapter (Table 2.1, 2.2, …). Caption is placed ABOVE the table, in Title Case. Maintain 1.5-line spacing before and after the table, and ensure it appears after its first mention in the text.

Figures — Numbered sequentially per chapter (Figure 2.1, 2.2, …). Caption is placed BELOW the figure, in Title Case, formatted as 'Figure 2.1: Title.' Maintain 1.5-line spacing around the caption.

Caption Alignment — A single-line caption is centred. A multi-line caption is left-aligned with a 2 cm hanging indent.

Landscape Pages & Copyright — Bind the TOP of landscape tables/figures toward the spine. Obtain permission for any table/figure reproduced from external sources (Appendix C) — verify licence terms even for open-access materials.

Examiner Tip

Every figure and table must be referenced in the body text BEFORE it appears — never insert a 'floating' table or figure with no surrounding discussion.

FORMATTING — APPLIES TO ALL CHAPTERS

Paragraphs, Lists & Page Layout

Spacing & Alignment — Use 1.5-line spacing throughout the report. All body paragraphs must be justified.

Paragraph Indentation — The first paragraph of each subsection aligns with the left margin; subsequent paragraphs are indented 1.5 cm (0.59 in).

Lists — Use lowercase Roman numerals (i, ii, iii, …). Numbering is indented 1.5 cm from the left margin; text uses a 1.25 cm hanging indent (2.75 cm total). Each list item ends with a full stop.

Equations — Centre-align equations and right-align the equation number, using tab stops. Maintain 1.5-line spacing before and after.

Page Breaks — Each new chapter starts on a new page. A subsection heading must never be the last line on a page — push it to the next page if needed.

Examiner Tip

These micro-formatting rules are exactly what examiners notice first. A report that looks professionally formatted builds credibility before the content is even read.

⚠ Common Mistakes to Avoid

✗ Objectives written as deliverables

'Develop a mobile app' is NOT an objective. Objectives must be measurable outcomes (what will be analysed, evaluated, or achieved — and by when).

✗ Design ≠ Implementation

If your implemented UI differs from your Figma mock-up, explain why. Unexplained inconsistencies are penalised.

✗ Literature review is just description

LR must compare, contrast, and critique — not just summarise. Every section must conclude with implications for your project.

✗ Incomplete data dictionary

Every field in every collection/table must appear. Partial data dictionaries are a recurring weakness.

✗ Agile claimed but not evidenced

If you declare Agile, show your sprints. A Gantt chart alone does not constitute Agile methodology.

✗ Testing lacks evidence

Showing 'Pass' without screenshots or actual output is insufficient. Provide evidence for all test types.

✗ Requirements not traced to findings

Requirements must originate from your questionnaire or interview data — not invented. Show the link explicitly.

✗ Exceeding the number of word

A common mistake is exceeding the word limit by including code snippets and their detailed explanations within the report.

Formatting & Submission Checklist

Document Formatting

- ✓ Times New Roman 12pt, 1.5 line spacing, justified text
- ✓ 2.5 cm margins; 1.5 cm paragraph indent (2nd+ paragraphs)
- ✓ Page numbers: Roman for prefatory pages, Arabic from Ch 1
- ✓ Each chapter starts on a new page

Referencing (CTR Harvard)

- ✓ CTR Harvard style for ALL in-text citations and references
- ✓ Minimum 20 references (within the past 5 years)
- ✓ No Wikipedia; avoid non-academic web sources
- ✓ Every in-text citation has a matching Reference List entry

Figures & Tables

- ✓ Numbered per chapter; caption above tables, below figures
- ✓ All figures/tables referenced in text before they appear
- ✓ Hanging indent for multi-line captions; landscape bound to spine
- ✓ Permission obtained for externally-sourced figures/tables (Appendix C)

Completeness & Declarations

- ✓ All 8 chapters present with all subsections
- ✓ Declaration: GenAI / vibe-coding tool usage boxes ticked if used
- ✓ Abstract (≤300 words, IMRaD) + Keywords + Subject Area present (FYP2)
- ✓ Appendices complete; Turnitin similarity within threshold

Key Takeaway

Every chapter tells a story.

Introduction sets the problem → Literature justifies your choices → Methodology shows your plan → Specification defines requirements → Design blueprints the solution → Implementation builds it → Testing validates it → Conclusion reflects on what was achieved.

Good luck with your FYP!